Data Wrangling with Dates in pandas

Quick guide · to_datetime · the .dt accessor · date arithmetic · resampling

Dates almost always arrive as **text**: from a CSV, an Excel export, an API or a form. Until you convert them, pandas cannot sort them in time order, subtract them, or group them by month. This short guide shows how to convert text to dates, pull out date parts, calculate with dates, and summarise data over time, first on a **Series** and then on a **DataFrame**.

Every example was run with pandas 3; the output is real.

Task	Tool
Convert text to dates	`pd.to_datetime(series, format=...)`, `read_csv(parse_dates=[...])`
Pull out parts (year, month, weekday…)	the `.dt` accessor: `s.dt.year`, `s.dt.day_name()`
Calculate with dates	subtraction gives a `Timedelta`; add `pd.Timedelta` or `pd.DateOffset`
Filter a period	comparisons, `.between()`, or `.loc["2026-10"]` on a date index
Summarise over time	`resample("W")`, `groupby(s.dt.month)`, `rolling(3)`

Part 1 Converting text to datetime

A Series of date strings

d1_basic.py

```
import pandas as pd

dates_text = pd.Series(["2026-10-01", "2026-10-15", "2026-11-03"])
print(dates_text.dtype)

dates = pd.to_datetime(dates_text)
print(dates)
```

OUTPUT

```
str
0   2026-10-01
1   2026-10-15
2   2026-11-03
dtype: datetime64[us]
```

Text has dtype `str` (or `object` in older pandas). After `pd.to_datetime` the dtype is `datetime64`: real dates that sort, subtract and group correctly.

Tell pandas the format

South African forms and spreadsheets usually write day first: `03/11/2026` is **3 November**, but by default pandas reads month first:

d1_format.py

```
import pandas as pd

sa_dates = pd.Series(["03/11/2026", "15/10/2026"])
print(pd.to_datetime(sa_dates, format="%d/%m/%Y"))
print(pd.to_datetime(pd.Series(["03/11/2026"])))
```

OUTPUT

```
0   2026-11-03
1   2026-10-15
dtype: datetime64[us]
0   2026-03-11
dtype: datetime64[us]
```

Without a format, `03/11/2026` became **11 March**: no error, just a wrong date. Always give `format=` (or `dayfirst=True`) when the format is known.

Code	Means	Example
`%Y` / `%y`	four-digit / two-digit year	2026 / 26
`%m`	month number	11
`%d`	day of month	03
`%b` / `%B`	short / full month name	Nov / November
`%H:%M:%S`	hours, minutes, seconds	14:30:00

d1_more_formats.py

```
import pandas as pd

print(pd.to_datetime(pd.Series(["3 Nov 2026", "15 Oct 2026"]), format="%d %b %Y"))
print(pd.to_datetime(pd.Series(["2026-11-03 14:30"]), format="%Y-%m-%d %H:%M"))
```

OUTPUT

```
0   2026-11-03
1   2026-10-15
dtype: datetime64[us]
0   2026-11-03 14:30:00
dtype: datetime64[us]
```

Values that will not convert

d1_coerce.py

```
import pandas as pd

messy = pd.Series(["2026-10-01", "not recorded", "2026-13-40", None])
print(pd.to_datetime(messy, format="%Y-%m-%d", errors="coerce"))
```

OUTPUT

```
0   2026-10-01
1          NaT
2          NaT
3          NaT
dtype: datetime64[us]
```

`errors="coerce"` turns anything unparseable into `NaT` ("not a time", the date version of `NaN`) instead of stopping. Then count and inspect the `NaT` rows; do not silently drop them.

Other sources

d1_parts.py

```
import pandas as pd

parts = pd.DataFrame({"year": [2026, 2026], "month": [10, 11], "day": [1, 3]})
print(pd.to_datetime(parts))

print(pd.to_datetime(pd.Series([1793520000]), unit="s"))
```

OUTPUT

```
0   2026-10-01
1   2026-11-03
dtype: datetime64[us]
0   2026-11-01 08:00:00
dtype: datetime64[s]
```

Separate year, month and day columns combine directly. System logs often store **Unix timestamps** (seconds since 1 January 1970): use `unit="s"`.

Convert while reading a CSV

d1_read_csv.py

```
import pandas as pd

orders = pd.read_csv("orders.csv", parse_dates=["order_date", "ship_date"])
print(orders.dtypes)
print(orders.head(3))
```

OUTPUT

```
order_id                 str
order_date    datetime64[us]
ship_date     datetime64[us]
branch                   str
amount               float64
dtype: object
  order_id order_date  ship_date    branch  amount
0     A101 2026-10-01 2026-10-03   Sandton  1250.0
1     A102 2026-10-01 2026-10-06  Rosebank   980.5
2     A103 2026-10-04 2026-10-05   Sandton  1430.0
```

Part 2 Pulling out date parts with .dt

On a datetime Series, the `.dt` accessor gives each date's parts, as a new Series you can store as a column:

d2_parts.py

```
orders["year"] = orders["order_date"].dt.year
orders["month"] = orders["order_date"].dt.month
orders["month_name"] = orders["order_date"].dt.month_name()
orders["weekday"] = orders["order_date"].dt.day_name()
orders["is_weekend"] = orders["order_date"].dt.dayofweek >= 5
print(orders[["order_id", "order_date", "month", "month_name", "weekday", "is_weekend"]])
```

OUTPUT

```
  order_id order_date  month month_name   weekday  is_weekend
0     A101 2026-10-01     10    October  Thursday       False
1     A102 2026-10-01     10    October  Thursday       False
2     A103 2026-10-04     10    October    Sunday        True
3     A104 2026-10-09     10    October    Friday       False
4     A105 2026-10-15     10    October  Thursday       False
5     A106 2026-10-22     10    October  Thursday       False
6     A107 2026-10-31     10    October  Saturday        True
7     A108 2026-11-03     11   November   Tuesday       False
```

Attribute	Gives
`.dt.year`, `.dt.month`, `.dt.day`	numbers
`.dt.quarter`	1 to 4
`.dt.dayofweek`	0 = Monday … 6 = Sunday
`.dt.day_name()`, `.dt.month_name()`	"Saturday", "October"
`.dt.is_month_end`	True on the last day of a month
`.dt.date`	the date without the time

Formatting dates as text for reports

d2_strftime.py

```
print(orders["order_date"].dt.strftime("%d %b %Y").head(3).tolist())
print(orders["order_date"].dt.strftime("%Y-%m").head(3).tolist())
```

OUTPUT

```
['01 Oct 2026', '01 Oct 2026', '04 Oct 2026']
['2026-10', '2026-10', '2026-10']
```

`strftime` turns dates back into text, so do it last, only for display.

Part 3 Calculating with dates

Differences: how long did shipping take?

d3_diff.py

```
orders["ship_time"] = orders["ship_date"] - orders["order_date"]
orders["ship_days"] = orders["ship_time"].dt.days
print(orders[["order_id", "order_date", "ship_date", "ship_time", "ship_days"]])
print("Average days to ship:", orders["ship_days"].mean())
```

OUTPUT

```
  order_id order_date  ship_date ship_time  ship_days
0     A101 2026-10-01 2026-10-03    2 days          2
1     A102 2026-10-01 2026-10-06    5 days          5
2     A103 2026-10-04 2026-10-05    1 days          1
3     A104 2026-10-09 2026-10-14    5 days          5
4     A105 2026-10-15 2026-10-16    1 days          1
5     A106 2026-10-22 2026-10-29    7 days          7
6     A107 2026-10-31 2026-11-02    2 days          2
7     A108 2026-11-03 2026-11-04    1 days          1
Average days to ship: 3.0
```

Subtracting two dates gives a **Timedelta** (a length of time). `.dt.days` turns it into a plain number for averages and charts.

Adding time: due dates and month ends

d3_add.py

```
orders["payment_due"] = orders["order_date"] + pd.Timedelta(days=30)
orders["month_end"] = orders["order_date"] + pd.offsets.MonthEnd(0)
orders["next_month"] = orders["order_date"] + pd.DateOffset(months=1)
print(orders[["order_date", "payment_due", "month_end", "next_month"]].head(4))
```

OUTPUT

```
  order_date payment_due  month_end next_month
0 2026-10-01  2026-10-31 2026-10-31 2026-11-01
1 2026-10-01  2026-10-31 2026-10-31 2026-11-01
2 2026-10-04  2026-11-03 2026-10-31 2026-11-04
3 2026-10-09  2026-11-08 2026-10-31 2026-11-09
```

`pd.Timedelta(days=30)` adds an exact number of days. `pd.DateOffset(months=1)` moves by calendar months, and `MonthEnd(0)` rolls forward to the end of the current month: useful for billing cycles and month-end reporting.

Filtering by date

d3_filter.py

```
october = orders[orders["order_date"].between("2026-10-01", "2026-10-31")]
print(len(october), "orders in October")

late = orders[orders["ship_days"] > 3]
print(late[["order_id", "branch", "ship_days"]])
```

OUTPUT

```
7 orders in October
  order_id    branch  ship_days
1     A102  Rosebank          5
3     A104    Soweto          5
5     A106   Sandton          7
```

Date columns compare directly with date strings such as `"2026-10-01"`. `between` includes both ends.

d3_today.py

```
today = pd.Timestamp("2026-11-05")
orders["days_since_order"] = (today - orders["order_date"]).dt.days
print(orders[["order_id", "order_date", "days_since_order"]].tail(3))
```

OUTPUT

```
  order_id order_date  days_since_order
5     A106 2026-10-22                14
6     A107 2026-10-31                 5
7     A108 2026-11-03                 2
```

In a real report use `pd.Timestamp.today()`. A fixed date keeps this example's output reproducible.

Part 4 Dates in a DataFrame: index, resample and rolling

A date index

d4_index.py

```
daily = orders.set_index("order_date").sort_index()
print(daily.loc["2026-10", ["order_id", "amount"]])
print(daily.loc["2026-10-20":"2026-11-30", "amount"].sum())
```

OUTPUT

```
            order_id   amount
order_date
2026-10-01     A101  1250.00
2026-10-01     A102   980.50
2026-10-04     A103  1430.00
2026-10-09     A104   610.25
2026-10-15     A105  1890.00
2026-10-22     A106   720.00
2026-10-31     A107  1105.75
2670.75
```

With dates as the index, `.loc["2026-10"]` selects a whole month and a date slice includes both ends. Sort the index first.

Resample: totals per week or month

d4_resample.py

```
print(daily["amount"].resample("W").sum())
print(daily["amount"].resample("ME").agg(["count", "sum"]))
```

OUTPUT

```
order_date
2026-10-04    3660.50
2026-10-11     610.25
2026-10-18    1890.00
2026-10-25     720.00
2026-11-01    1105.75
2026-11-08     845.00
Freq: W-SUN, Name: amount, dtype: float64
            count     sum
order_date
2026-10-31      7  7986.5
2026-11-30      1   845.0
```

`resample` is a `groupby` for time: `"D"` day, `"W"` week (ending Sunday), `"ME"` month end, `"QE"` quarter end. Empty weeks still appear (with 0), which keeps a time chart honest.

> **Watch out:** Older tutorials write `resample("M")`. Recent pandas renamed the month and quarter aliases to `"ME"` and `"QE"`; the old ones are deprecated or removed.

Group by a date part

d4_groupby.py

```
print(orders.groupby(orders["order_date"].dt.month_name())["amount"].sum())
print(orders.groupby(["branch", orders["order_date"].dt.month])["amount"].sum())
```

OUTPUT

```
order_date
November     845.0
October     7986.5
Name: amount, dtype: float64
branch    order_date
Rosebank  10            2870.5
          11             845.0
Sandton   10            3400.0
Soweto    10            1716.0
Name: amount, dtype: float64
```

Month names sort alphabetically (November before October). Group by `.dt.month`, or by `.dt.to_period("M")`, to keep calendar order.

Change over time: shift, diff and rolling

d4_rolling.py

```
weekly = daily["amount"].resample("W").sum().to_frame("sales")
weekly["previous_week"] = weekly["sales"].shift(1)
weekly["change"] = weekly["sales"].diff()
weekly["rolling_2wk"] = weekly["sales"].rolling(2).mean()
print(weekly.round(2))
```

OUTPUT

```
              sales  previous_week   change  rolling_2wk
order_date
2026-10-04  3660.50            NaN      NaN          NaN
2026-10-11   610.25        3660.50 -3050.25      2135.38
2026-10-18  1890.00         610.25  1279.75      1250.12
2026-10-25   720.00        1890.00 -1170.00      1305.00
2026-11-01  1105.75         720.00   385.75       912.88
2026-11-08   845.00        1105.75  -260.75       975.38
```

`shift(1)` brings last period's value onto this row; `diff()` is this period minus the last; `rolling(2).mean()` averages the latest two periods to smooth out noise. The first rows are `NaN` because there is no earlier period to use.

Time zones (one line each)

d4_tz.py

```
utc_times = pd.Series(pd.to_datetime(["2026-11-03 07:30", "2026-11-03
16:00"])).dt.tz_localize("UTC")
print(utc_times.dt.tz_convert("Africa/Johannesburg"))
```

OUTPUT

```
0   2026-11-03 09:30:00+02:00
1   2026-11-03 18:00:00+02:00
dtype: datetime64[us, Africa/Johannesburg]
```

`tz_localize` says which zone a time is in; `tz_convert` shows it in another. APIs and logs usually use UTC.

Try it yourself

TRY IT YOURSELF

Using `orders.csv`: (a) read it with both date columns parsed; (b) add `order_weekday` and `ship_days`; (c) which branch has the longest average shipping time? (d) total the amount per month with `resample`; (e) list the orders placed on a weekend.

Answers

d5_answers.py

```
import pandas as pd

orders = pd.read_csv("orders.csv", parse_dates=["order_date", "ship_date"])
orders["order_weekday"] = orders["order_date"].dt.day_name()
orders["ship_days"] = (orders["ship_date"] - orders["order_date"]).dt.days

print(orders.groupby("branch")["ship_days"].mean().sort_values(ascending=False))
print(orders.set_index("order_date")["amount"].resample("ME").sum())
print(orders.loc[orders["order_date"].dt.dayofweek >= 5, ["order_id", "order_date",
"order_weekday"]])
```

OUTPUT

```
branch
Soweto      3.500000
Sandton     3.333333
Rosebank    2.333333
Name: ship_days, dtype: float64
order_date
2026-10-31    7986.5
2026-11-30     845.0
Freq: ME, Name: amount, dtype: float64
  order_id order_date order_weekday
2     A103 2026-10-04        Sunday
6     A107 2026-10-31      Saturday
```

Summary

Habit	Why
Convert dates as soon as you load	text dates sort and compare wrongly
Always state `format=` for day-first dates	03/11 can silently become 11 March
Use `errors="coerce"`, then inspect `NaT`	find bad values instead of crashing or hiding them
Format with `strftime` only for display	text dates cannot be calculated with
Use `resample` for time series	it includes empty periods, unlike a plain `groupby`